

LAB-12.5: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

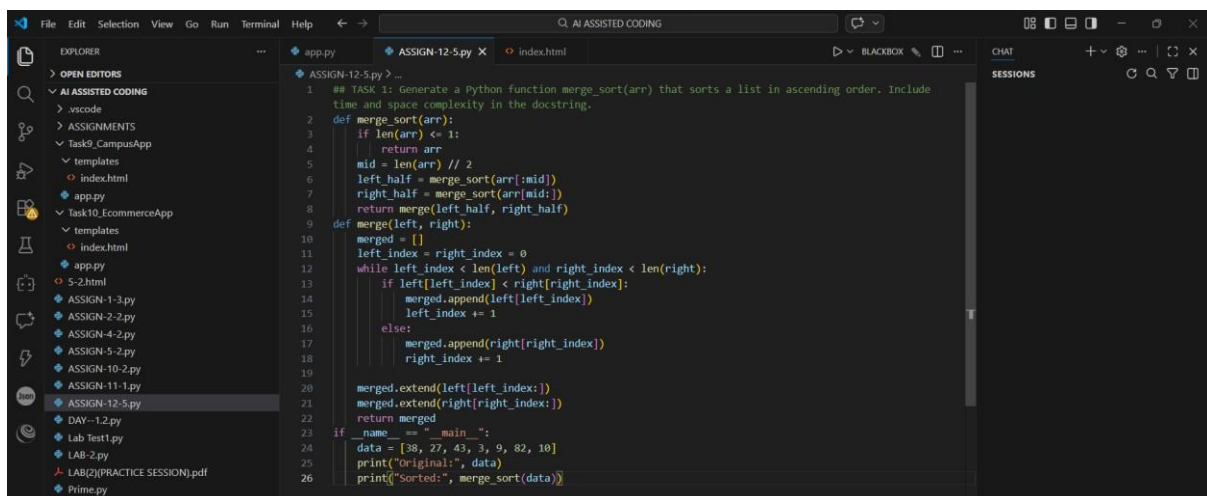
Harika Jupaka

2403A51L31

B-52

Task 1 – Merge Sort:

Prompt: Generate a Python function `merge_sort(arr)` that sorts a list in ascending order. Include time and space complexity in the docstring.



```
1  ## TASK 1: Generate a Python function merge_sort(arr) that sorts a list in ascending order. Include
2  time and space complexity in the docstring.
3  def merge_sort(arr):
4      if len(arr) <= 1:
5          return arr
6      mid = len(arr) // 2
7      left_half = merge_sort(arr[:mid])
8      right_half = merge_sort(arr[mid:])
9      return merge(left_half, right_half)
10 def merge(left, right):
11     merged = []
12     left_index = right_index = 0
13     while left_index < len(left) and right_index < len(right):
14         if left[left_index] < right[right_index]:
15             merged.append(left[left_index])
16             left_index += 1
17         else:
18             merged.append(right[right_index])
19             right_index += 1
20     merged.extend(left[left_index:])
21     merged.extend(right[right_index:])
22     return merged
23 if __name__ == "__main__":
24     data = [38, 27, 43, 3, 9, 82, 10]
25     print("Original:", data)
26     print("Sorted:", merge_sort(data))
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python
.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-12-5.py"
Original: [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]
```

Explanation: Merge Sort is chosen because it guarantees $O(n \log n)$ time complexity in all cases. It is stable and efficient for large datasets.

Task 2 – Binary Search:

Prompt: Create a Python function `binary_search(arr, target)` that returns the index of the target or -1 if not found. Include best, average, and worst-case complexities in the docstring.

OUTPUT:

Explanation: Binary Search works on sorted lists and reduces the search space by half each step. It provides $O(\log n)$ efficiency, making it faster than linear search.

Task 3 – Healthcare Appointment System:

Prompt: Suggest efficient searching and sorting algorithms for managing appointment records and implement them in Python.

```
75 def binary_search_appointments(appointments, target_date):
76     left, right = 0, len(appointments) - 1
77     while left <= right:
78         mid = left + (right - left) // 2
79         if appointments[mid].date == target_date:
80             return mid
81         elif appointments[mid].date < target_date:
82             left = mid + 1
83         else:
84             right = mid - 1
85     return -1
86 if __name__ == "__main__":
87     appointments = [
88         Appointment("2024-07-01", "10:00", "Doctor's appointment"),
89         Appointment("2024-07-02", "14:00", "Meeting with client"),
90         Appointment("2024-07-01", "12:00", "Lunch with friend"),
91     ]
92     sorted_appointments = merge_sort_appointments(appointments)
93     target_date = "2024-07-01"
94     index = binary_search_appointments(sorted_appointments, target_date)
95     if index != -1:
96         print(f"Appointment found: {sorted_appointments[index].description} at {sorted_appointments[index].time}")
97     else:
98         print("No appointment found on the target date.")
```

OUTPUT:

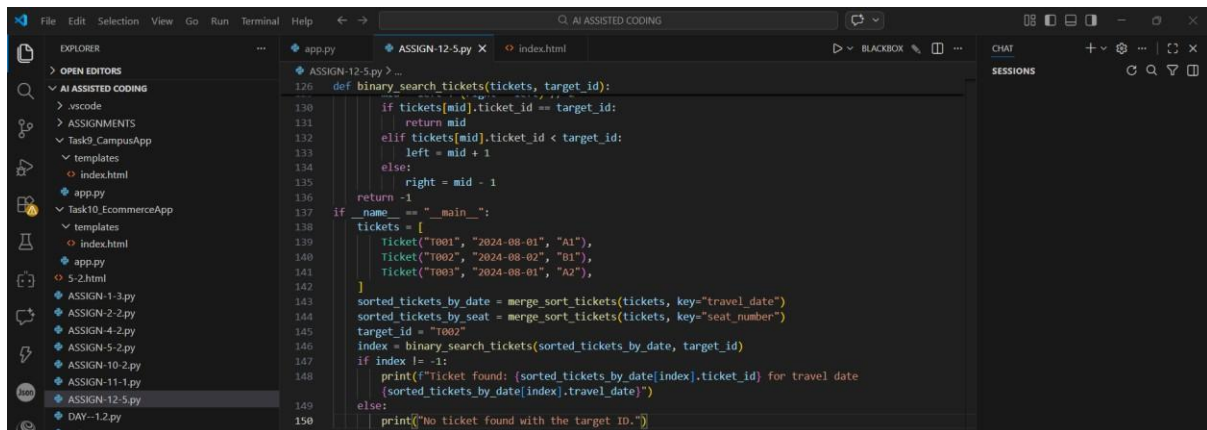
```
Appointment found: Lunch with friend at 12:00
```

Explanation: Binary Search is used for fast appointment ID lookup. Merge Sort is used for stable and efficient sorting by time or consultation fee.

Task 4 – Railway Reservation System:

Prompt: Recommend suitable algorithms to search tickets by ticket ID and sort bookings by travel date or seat number.

```
100 ## TASK 4: Recommend suitable algorithms to search tickets by ticket ID and sort bookings by travel
101 date or seat number.
102 class Ticket:
103     def __init__(self, ticket_id, travel_date, seat_number):
104         self.ticket_id = ticket_id
105         self.travel_date = travel_date
106         self.seat_number = seat_number
107 def merge_sort_tickets(tickets, key):
108     if len(tickets) <= 1:
109         return tickets
110     mid = len(tickets) // 2
111     left_half = merge_sort_tickets(tickets[:mid], key)
112     right_half = merge_sort_tickets(tickets[mid:], key)
113     return merge_tickets(left_half, right_half, key)
114 def merge_tickets(left, right, key):
115     merged = []
116     left_index = right_index = 0
117     while left_index < len(left) and right_index < len(right):
118         if getattr(left[left_index], key) < getattr(right[right_index], key):
119             merged.append(left[left_index])
120             left_index += 1
121         else:
122             merged.append(right[right_index])
123             right_index += 1
124     merged.extend(left[left_index:])
125     merged.extend(right[right_index:])
126     return merged
127 def binary_search_tickets(tickets, target_id):
128     left, right = 0, len(tickets) - 1
129     while left <= right:
130         mid = left + (right - left) // 2
131         if tickets[mid].ticket_id == target_id:
132             return mid
133         elif tickets[mid].ticket_id < target_id:
134             left = mid + 1
135         else:
```



```
126 def binary_search_tickets(tickets, target_id):
127     left = 0
128     right = len(tickets) - 1
129     while left <= right:
130         mid = (left + right) // 2
131         if tickets[mid].ticket_id == target_id:
132             return mid
133         elif tickets[mid].ticket_id < target_id:
134             left = mid + 1
135         else:
136             right = mid - 1
137     return -1
138
139 if __name__ == "__main__":
140     tickets = [
141         Ticket("T001", "2024-08-01", "A1"),
142         Ticket("T002", "2024-08-02", "B1"),
143         Ticket("T003", "2024-08-01", "A2"),
144     ]
145     sorted_tickets_by_date = merge_sort_tickets(tickets, key="travel_date")
146     sorted_tickets_by_seat = merge_sort_tickets(tickets, key="seat_number")
147     target_id = "1002"
148     index = binary_search_tickets(sorted_tickets_by_date, target_id)
149     if index != -1:
150         print(f"Ticket found: {sorted_tickets_by_date[index].ticket_id} for travel date {sorted_tickets_by_date[index].travel_date}")
151     else:
152         print("No ticket found with the target ID.")
```

OUTPUT:

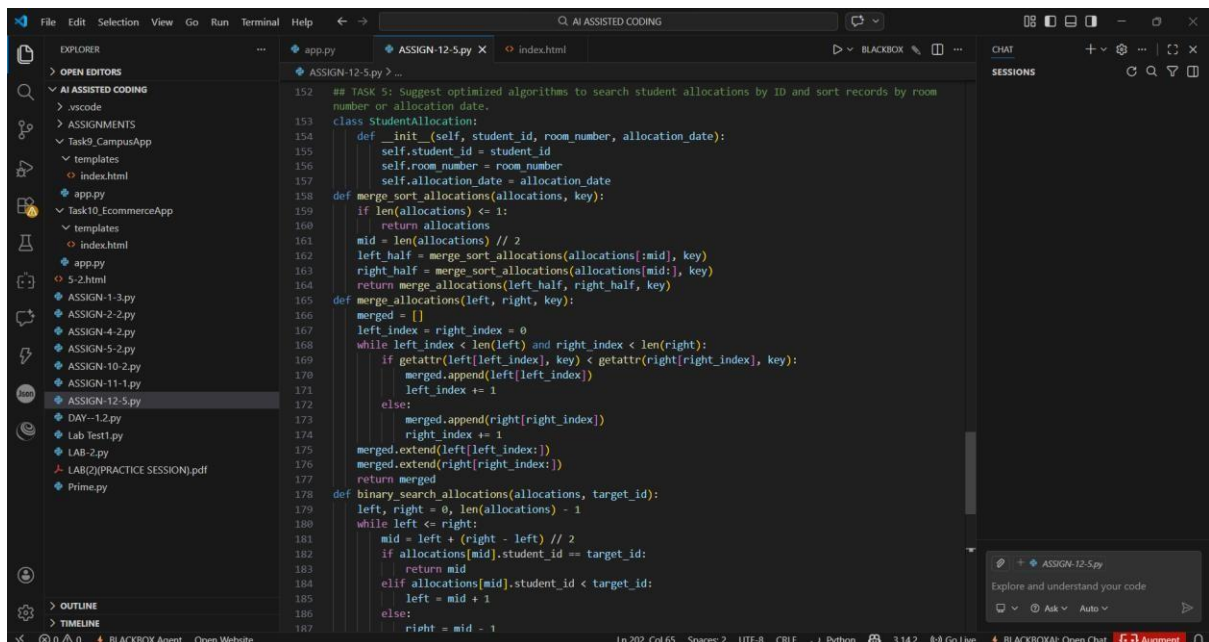


```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python
Ticket found: 1002 for travel date 2024-08-02
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation: Binary Search ensures fast ticket lookup in sorted records. Merge Sort guarantees consistent $O(n \log n)$ sorting performance.

Task 5 – Hostel Room Allocation:

Prompt: Suggest optimized algorithms to search student allocations by ID and sort records by room number or allocation date.



```
152 ## TASK 5: Suggest optimized algorithms to search student allocations by ID and sort records by room
153 number or allocation date.
154 class StudentAllocation:
155     def __init__(self, student_id, room_number, allocation_date):
156         self.student_id = student_id
157         self.room_number = room_number
158         self.allocation_date = allocation_date
159
160 def merge_sort_allocations(allocations, key):
161     if len(allocations) <= 1:
162         return allocations
163     mid = len(allocations) // 2
164     left_half = merge_sort_allocations(allocations[:mid], key)
165     right_half = merge_sort_allocations(allocations[mid:], key)
166     return merge_allocations(left_half, right_half, key)
167
168 def merge_allocations(left, right, key):
169     merged = []
170     left_index = right_index = 0
171     while left_index < len(left) and right_index < len(right):
172         if getattr(left[left_index], key) < getattr(right[right_index], key):
173             merged.append(left[left_index])
174             left_index += 1
175         else:
176             merged.append(right[right_index])
177             right_index += 1
178     merged.extend(left[left_index:])
179     merged.extend(right[right_index:])
180     return merged
181
182 def binary_search_allocations(allocations, target_id):
183     left, right = 0, len(allocations) - 1
184     while left <= right:
185         mid = (left + right) // 2
186         if allocations[mid].student_id == target_id:
187             return mid
188         elif allocations[mid].student_id < target_id:
189             left = mid + 1
190         else:
191             right = mid - 1
```



```
178 def binary_search_allocations(allocations, target_id):
184     elif allocations[mid].student_id < target_id:
185         left = mid + 1
186     else:
187         right = mid - 1
188     return -1
189 if __name__ == "__main__":
190     allocations = [
191         StudentAllocation("S001", "101", "2024-09-01"),
192         StudentAllocation("S002", "102", "2024-09-02"),
193         StudentAllocation("S003", "101", "2024-09-01"),
194     ]
195     sorted_allocations_by_room = merge_sort_allocations(allocations, key="room_number")
196     sorted_allocations_by_date = merge_sort_allocations(allocations, key="allocation_date")
197     target_id = "S002"
198     index = binary_search_allocations(sorted_allocations_by_room, target_id)
199     if index != -1:
200         print(f"Student allocation found: (sorted_allocations_by_room[index].student_id in room
201             (sorted_allocations_by_room[index].room_number)")
202     else:
203         print("No student allocation found with the target ID.")
```

OUTPUT:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> .\ASSIGN-12-5.py
Student allocation found: S002 in room 102
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation: Binary Search enables efficient student ID searching. Merge Sort handles large room allocation records reliably.

Task 6 – Movie Streaming Platform:

Prompt: Recommend searching and sorting algorithms for managing movie records by ID, rating, and release year.

```
204 ## TASK 6: Recommend searching and sorting algorithms for managing movie records by ID, rating, and
205 release year.
206 class Movie:
207     def __init__(self, movie_id, rating, release_year):
208         self.movie_id = movie_id
209         self.rating = rating
210         self.release_year = release_year
211 def merge_sort_movies(movies, key):
212     if len(movies) <= 1:
213         return movies
214     mid = len(movies) // 2
215     left_half = merge_sort_movies(movies[:mid], key)
216     right_half = merge_sort_movies(movies[mid:], key)
217     return merge_movies(left_half, right_half, key)
218 def merge_movies(left, right, key):
219     merged = []
220     left_index = right_index = 0
221     while left_index < len(left) and right_index < len(right):
222         if getattr(left[left_index], key) < getattr(right[right_index], key):
223             merged.append(left[left_index])
224             left_index += 1
225         else:
226             merged.append(right[right_index])
227             right_index += 1
228     merged.extend(left[left_index:])
229     merged.extend(right[right_index:])
230     return merged
231 def binary_search_movies(movies, target_id):
232     left, right = 0, len(movies) - 1
233     while left <= right:
234         mid = left + (right - left) // 2
235         if movies[mid].movie_id == target_id:
236             return mid
237         elif movies[mid].movie_id < target_id:
238             left = mid + 1
239         else:
240             right = mid - 1
```

```
File Edit Selection View Go Run Terminal Help
AI ASSISTED CODING

EXPLORER
  OPEN EDITORS
  AI ASSISTED CODING
  .vscode
  ASSIGNMENTS
  Task9_CampusApp
  templates
  index.html
  app.py
  Task10_EcommerceApp
  templates
  index.html
  app.py
  5-2.html
  ASSIGN-1-3.py
  ASSIGN-2-2.py
  ASSIGN-4-2.py
  ASSIGN-5-2.py
  ASSIGN-10-2.py
  ASSIGN-11-1.py
  ASSIGN-12-5.py
  DAY-1-2.py
  Lab Test1.py

ASSIGN-12-5.py
230 def binary_search_movies(movies, target_id):
231     if movies[mid].movie_id == target_id:
232         return mid
233     elif movies[mid].movie_id < target_id:
234         left = mid + 1
235     else:
236         right = mid - 1
237     return -1
238
239 if __name__ == "__main__":
240     movies = [
241         Movie("M001", 8.5, 2020),
242         Movie("M002", 7.2, 2019),
243         Movie("M003", 9.0, 2021),
244     ]
245     sorted_movies_by_rating = merge_sort_movies(movies, key="rating")
246     sorted_movies_by_year = merge_sort_movies(movies, key="release_year")
247     target_id = "M002"
248     index = binary_search_movies(sorted_movies_by_rating, target_id)
249     if index != -1:
250         print(f"Movie found: {sorted_movies_by_rating[index].movie_id} with rating {sorted_movies_by_rating[index].rating}")
251     else:
252         print("No movie found with the target ID.")
```

OUTPUT:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & c:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python
one "c:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING\ASSIGN-12-5.py"
* No movie found with the target ID.
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation: Binary Search quickly locates movies by ID. Merge Sort efficiently sorts movies by rating or release year.

Task 7 – Smart Agriculture System:

Prompt: Suggest appropriate algorithms to search crop details by ID and sort crops by moisture level or yield estimate.

```
File Edit Selection View Go Run Terminal Help
AI ASSISTED CODING

EXPLORER
  OPEN EDITORS
  AI ASSISTED CODING
  .vscode
  ASSIGNMENTS
  Task9_CampusApp
  templates
  index.html
  app.py
  Task10_EcommerceApp
  templates
  index.html
  app.py
  5-2.html
  ASSIGN-1-3.py
  ASSIGN-2-2.py
  ASSIGN-4-2.py
  ASSIGN-5-2.py
  ASSIGN-10-2.py
  ASSIGN-11-1.py
  ASSIGN-12-5.py
  DAY-1-2.py
  Lab Test1.py
  Lab Test2.py
  LAB2(PRACTICE SESSION).pdf
  Prime.py

ASSIGN-12-5.py
256 """ TASK 7: Suggest appropriate algorithms to search crop details by ID and sort crops by moisture level
257 or yield estimate.
258 class Crop:
259     def __init__(self, crop_id, moisture_level, yield_estimate):
260         self.crop_id = crop_id
261         self.moisture_level = moisture_level
262         self.yield_estimate = yield_estimate
263
264 def merge_sort_crops(crops, key):
265     if len(crops) <= 1:
266         return crops
267     mid = len(crops) // 2
268     left_half = merge_sort_crops(crops[:mid], key)
269     right_half = merge_sort_crops(crops[mid:], key)
270     return merge_crops(left_half, right_half, key)
271
272 def merge_crops(left, right, key):
273     merged = []
274     left_index = right_index = 0
275     while left_index < len(left) and right_index < len(right):
276         if getattr(left[left_index], key) < getattr(right[right_index], key):
277             merged.append(left[left_index])
278             left_index += 1
279         else:
280             merged.append(right[right_index])
281             right_index += 1
282     merged.extend(left[left_index:])
283     merged.extend(right[right_index:])
284     return merged
285
286 def binary_search_crops(crops, target_id):
287     left, right = 0, len(crops) - 1
288     while left <= right:
289         mid = left + (right - left) // 2
290         if crops[mid].crop_id == target_id:
291             return mid
292         elif crops[mid].crop_id < target_id:
293             left = mid + 1
294         else:
295             right = mid - 1
```

```
282 def binary_search_crops(crops, target_id):
283     left = 0
284     right = len(crops) - 1
285     while left <= right:
286         mid = (left + right) // 2
287         if crops[mid].crop_id < target_id:
288             left = mid + 1
289         elif crops[mid].crop_id > target_id:
290             right = mid - 1
291         else:
292             return mid
293     return -1
294
295 if __name__ == "__main__":
296     crops = [
297         Crop("C001", 20.5, 1000),
298         Crop("C002", 15.0, 800),
299         Crop("C003", 25.0, 1200),
300     ]
301     sorted_crops_by_moisture = merge_sort_crops(crops, key="moisture_level")
302     sorted_crops_by_yield = merge_sort_crops(crops, key="yield_estimate")
303     target_id = "C002"
304     index = binary_search_crops(sorted_crops_by_moisture, target_id)
305     if index != -1:
306         print(f"Crop found: {sorted_crops_by_moisture[index].crop_id} with moisture level {sorted_crops_by_moisture[index].moisture_level}")
307     else:
308         print("No crop found with the target ID.")
```

OUTPUT:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> python C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING\ASSIGN-12-5.py
Crop found: C002 with moisture level 15.0
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation: Binary Search improves search efficiency in sorted crop data. Merge Sort ensures accurate sorting of agricultural metrics.

Task 8 – Airport Flight Management:

Prompt: Recommend algorithms to search flight records by flight ID and sort flights by departure or arrival time.

```
308 ## TASK 8: Recommend algorithms to search flight records by flight ID and sort flights by departure or arrival time.
309 class Flight:
310     def __init__(self, flight_id, departure_time, arrival_time):
311         self.flight_id = flight_id
312         self.departure_time = departure_time
313         self.arrival_time = arrival_time
314
315 def merge_sort_flights(flights, key):
316     if len(flights) <= 1:
317         return flights
318     mid = len(flights) // 2
319     left_half = merge_sort_flights(flights[:mid], key)
320     right_half = merge_sort_flights(flights[mid:], key)
321     return merge_flights(left_half, right_half, key)
322
323 def merge_flights(left, right, key):
324     merged = []
325     left_index = right_index = 0
326     while left_index < len(left) and right_index < len(right):
327         if getattr(left[left_index], key) < getattr(right[right_index], key):
328             merged.append(left[left_index])
329             left_index += 1
330         else:
331             merged.append(right[right_index])
332             right_index += 1
333     merged.extend(left[left_index:])
334     merged.extend(right[right_index:])
335     return merged
336
337 def binary_search_flights(flights, target_id):
338     left, right = 0, len(flights) - 1
339     while left <= right:
340         mid = (left + right) // 2
341         if flights[mid].flight_id == target_id:
342             return mid
343         elif flights[mid].flight_id < target_id:
344             left = mid + 1
345         else:
346             right = mid - 1
```

```
334 def binary_search_flights(flights, target_id):
340     elif flights[mid].flight_id < target_id:
341         left = mid + 1
342     else:
343         right = mid - 1
344     return -1
345 if __name__ == "__main__":
346     flights = [
347         Flight("F001", "2024-10-01 08:00", "2024-10-01 12:00"),
348         Flight("F002", "2024-10-01 09:00", "2024-10-01 13:00"),
349         Flight("F003", "2024-10-01 07:00", "2024-10-01 11:00"),
350     ]
351     sorted_flights_by_departure = merge_sort_flights(flights, key="departure_time")
352     sorted_flights_by_arrival = merge_sort_flights(flights, key="arrival_time")
353     target_id = "F002"
354     index = binary_search_flights(sorted_flights_by_departure, target_id)
355     if index != -1:
356         print(f"Flight found: {sorted_flights_by_departure[index].flight_id} departing at
357             {sorted_flights_by_departure[index].departure_time}")
358     else:
359         print("No flight found with the target ID.")
```

OUTPUT:

```
318 left_half = merge_sort_flights(flights[:mid], key)
319 right_half = merge_sort_flights(flights[mid:], key)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AUGMENT NEXT EDIT TRUFFLE
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python
.exe "c:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING\ASSIGN-12-5.py"
Flight found: F002 departing at 2024-10-01 09:00
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation: Binary Search provides fast flight ID lookup. Merge Sort ensures stable sorting of time-based flight records.